

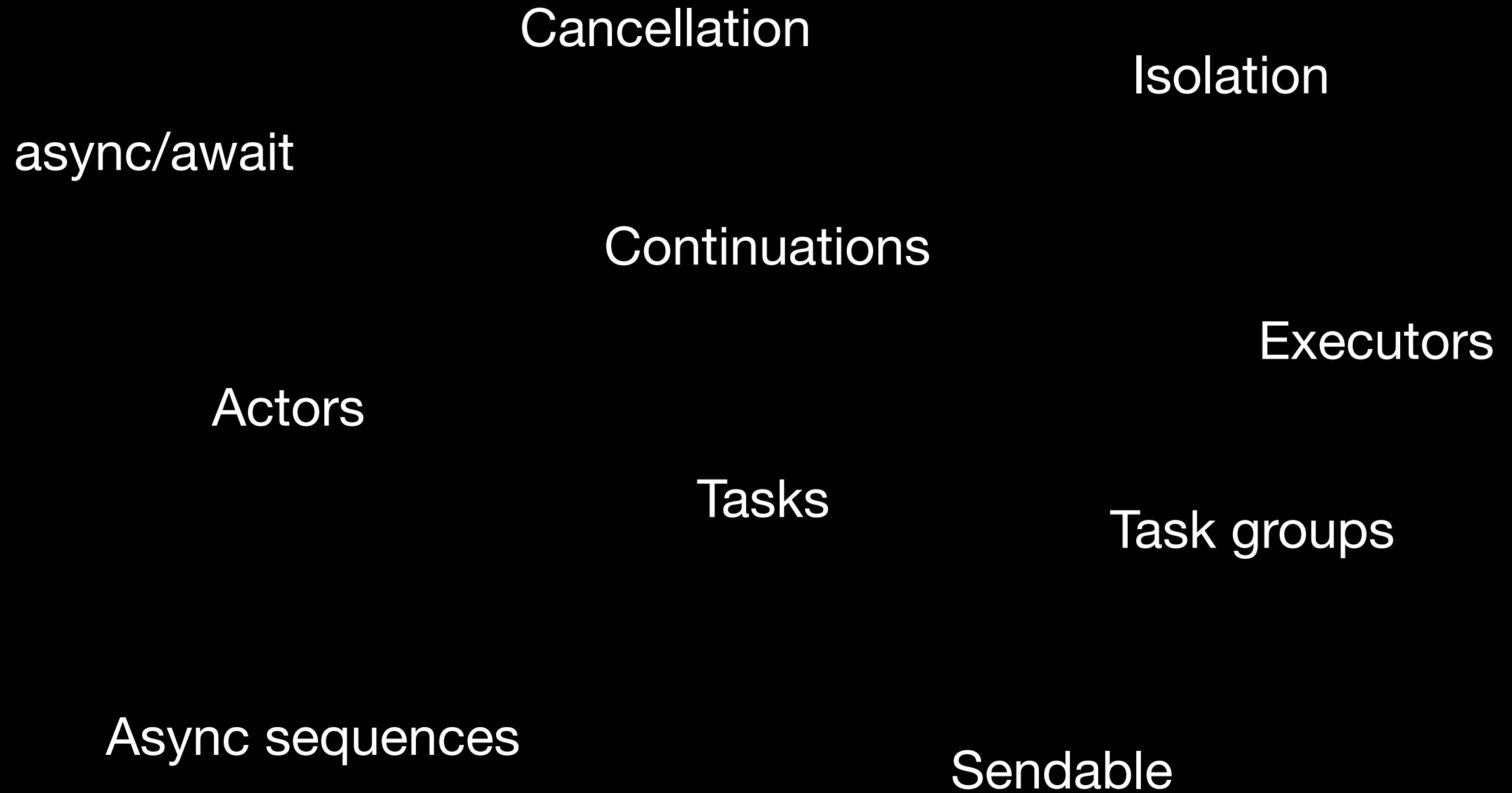
Designing APIs for Swift Concurrency

CocoaHeads / SLUG February 2026

Designing APIs for Swift Concurrency

CocoaHeads / SLUG February 2026

Swift Concurrency



Swift Concurrency

Cancellation

async/await

Isolation

Continuations

Executors

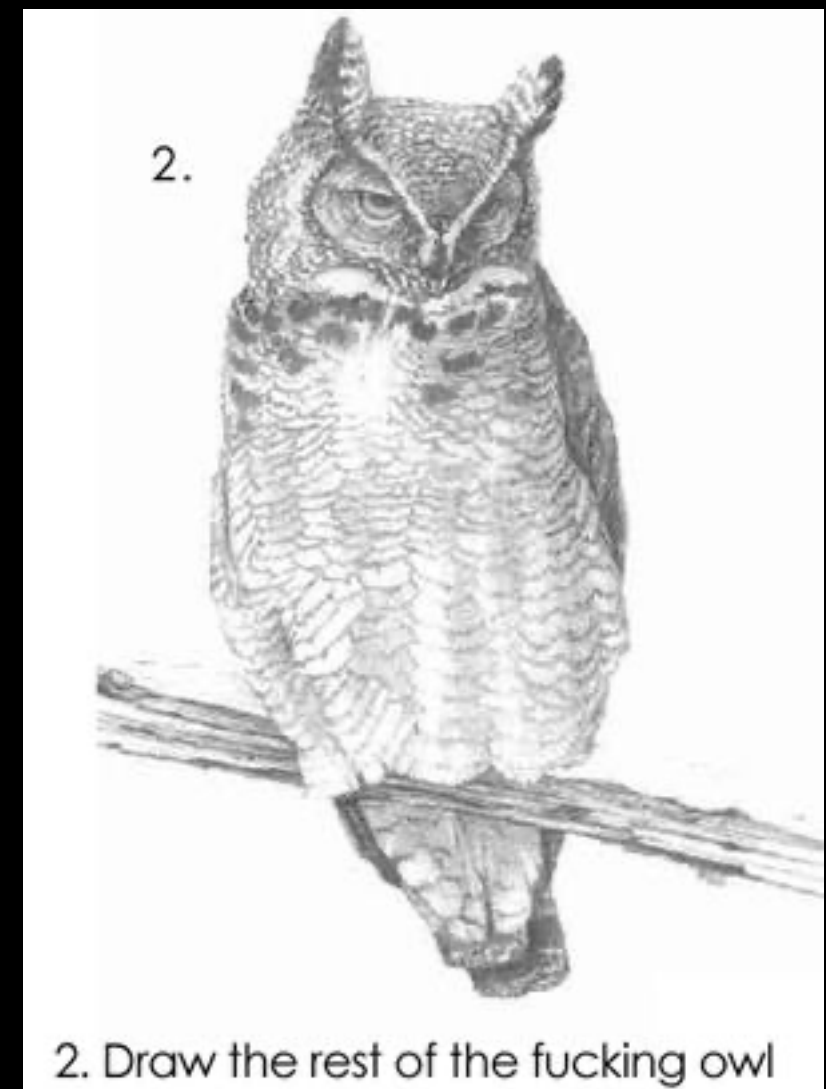
Actors

Tasks

Async sequences

Task groups

Sendable



Swift Concurrency

Swift Concurrency by example

- This talk is not:
 - ...a concept or API overview
 - ...help appeasing the compiler
- This talk is:
 - ...focused on correctness and ergonomics
 - ...strategies to analyze concurrent code
- Disclaimers
 - Sequences abridged
 - Claude Saagar can make mistakes

Connection Negotiation

Connection Negotiation

System API

```
protocol Client {  
    static func connect(  
        remote: Remote,  
        completion: @escaping (Result<Connection, Error>) -> Void)  
}
```

Connection Negotiation

async API

```
protocol Client {  
    static func connect(  
        remote: Remote,  
        completion: @escaping (Result<Connection, Error>) -> Void)  
}  
  
extension Client {  
    static func connect(remote: Remote) async throws -> Connection  
}
```

Connection Negotiation

async API

```
protocol Client {  
    static func connect(  
        remote: Remote,  
        completion: @escaping (Result<Connection, Error>) -> Void)  
}  
  
extension Client {  
    static func connect(remote: Remote) async throws -> Connection  
}
```

Connection Negotiation

async API

```
protocol Client {  
    static func connect(  
        remote: Remote,  
        completion: @escaping (Result<Connection, Error>) -> Void)  
}  
  
extension Client {  
    static func connect(remote: Remote) async throws -> Connection  
  
}
```

Connection Negotiation

async API

```
protocol Client {
    static func connect(
        remote: Remote,
        completion: @escaping (Result<Connection, Error>) -> Void)
}

extension Client {
    static func connect(remote: Remote) async throws -> Connection {
        try await withCheckedThrowingContinuation { continuation in
            Self.connect(remote: remote) {
                continuation.resume(with: $0)
            }
        }
    }
}
```

Connection Negotiation

async API

```
protocol Client {
    static func connect(
        remote: Remote,
        completion: @escaping (Result<Connection, Error>) -> Void)
}

extension Client {
    static func connect(remote: Remote) async throws -> Connection {
        try await withCheckedThrowingContinuation { continuation in
            Self.connect(remote: remote) {
                continuation.resume(with: $0)
            }
        }
    }
}
```

Connection Negotiation

async API

```
protocol Client {
```

```
}
```

Connection Negotiation

async API

```
protocol Client {
```

```
}
```

Connection Negotiation

New system API

```
protocol Client {  
    init(callback: @escaping (Result<Connection, Error>) -> Void)  
    func connect(remote: Remote)  
    func cancel()  
}
```

Connection Negotiation

async API

```
protocol Client {
    init(callback: @escaping (Result<Connection, Error>) -> Void)
    func connect(remote: Remote)
    func cancel()
}

extension Client {
    static func connect(remote: Remote) async throws -> Connection {
        try await withCheckedThrowingContinuation { continuation in
            let client = Client {
                continuation.resume(with: $0)
            }
            client.connect(remote: remote)
        }
    }
}
```

Connection Negotiation

async API

```
protocol Client {
    init(callback: @escaping (Result<Connection, Error>) -> Void)
    func connect(remote: Remote)
    func cancel()
}

extension Client {
    static func connect(remote: Remote) async throws -> Connection {
        try await withCheckedThrowingContinuation { continuation in
            let client = Client {
                continuation.resume(with: $0)
            }
            client.connect(remote: remote)
        }
    }
}
```

Connection Negotiation

async API with cancellation

```
protocol Client {
    init(callback: @escaping (Result<Connection, Error>) -> Void)
    func connect(remote: Remote)
    func cancel()
}

extension Client {
    static func connect(remote: Remote) async throws -> Connection {
        try await withCheckedThrowingContinuation { continuation in
            let client = Client {
                continuation.resume(with: $0)
            }

            client.connect(remote: remote)
        }
    }
}
```

Connection Negotiation

async API with cancellation

```
protocol Client {
    init(callback: @escaping (Result<Connection, Error>) -> Void)
    func connect(remote: Remote)
    func cancel()
}

extension Client {
    static func connect(remote: Remote) async throws -> Connection {
        try await withCheckedThrowingContinuation { continuation in
            let client = Client {
                continuation.resume(with: $0)
            }
            withTaskCancellationHandler(operation: {
                client.connect(remote: remote)
            }, onCancel: {
                client.cancel()
                continuation.resume(throwing: CancellationError())
            })
        }
    }
}
```

Connection Negotiation

Cancellation correctness

Documentation

Language: Swift

< All Technologies

Swift

func withTaskCancellationHan...

func withTaskCancellationHa...

Suspending Execution

static func yield() async

static func sleep(nanoseconds...

static func sleep<C>(for: C.Ins...

static func sleep<C>(until: C.In...

Escalating Tasks

func escalatePriority(to: TaskP...

Comparing Tasks

static func == (Task<Success, ...

static func != (Self, Self) -> Bool

var hashValue: Int

func hash(into: inout Hasher)

Filter

/

developer.apple.com/documentation/swift/withtaskcancellationhandler(ope

Swift / withTaskCancellationHandler(operation:onCancel:isolation:)

Function

withTaskCancellationHandler(operation:onCancel:isolation:)

Execute an operation with a cancellation handler that's immediately invoked if the current task is canceled.

iOS 13.0+ | iPadOS 13.0+ | Mac Catalyst 13.0+ | macOS 10.15+ | tvOS 13.0+ | visionOS 1.0+ | watchOS 6.0+

```
@backDeployed(before: macOS 15.0, iOS 18.0, watchOS 11.0, tvOS 18.0, visionOS 2.0)
func withTaskCancellationHandler<T>(
    operation: () async throws -> T,
    onCancel handler: @Sendable () -> Void,
    isolation: isolated (any Actor)? = #isolation
) async rethrows -> T
```

Parameters

Swift / withTaskCancellationHandler(operation:onCancel:isolation:)

Function

withTaskCancellationHandler(operation:onCancel:isolation:)

Execute an operation with a cancellation handler that's immediately invoked if the current task is canceled.

iOS 13.0+ | iPadOS 13.0+ | Mac Catalyst 13.0+ | macOS 10.15+ | tvOS 13.0+ | visionOS 1.0+ | watchOS 6.0+

```
@backDeployed(before: macOS 15.0, iOS 18.0, watchOS 11.0, tvOS 18.0, visionOS 2.0)
func withTaskCancellationHandler<T>(
    operation: () async throws -> T,
    onCancel handler: @Sendable () -> Void,
    isolation: isolated (any Actor)? = #isolation
) async rethrows -> T
```

Parameters

The operation to perform.

handler

A closure to execute on cancellation. If the task is canceled, this closure is called at most once; otherwise, it isn't called.

isolation

The actor that the operation and cancellation handler are isolated to.

Discussion

This differs from the operation cooperatively checking for cancellation and reacting to it in that the cancellation handler is *always* and *immediately* invoked when the task is canceled. For example, even if the operation is running code that never checks for cancellation, a cancellation handler still runs and provides a chance to run some cleanup code:

```
await withTaskCancellationHandler {  
    var sum = 0  
    while condition {  
        sum += 1  
    }  
    return sum  
} onCancel: {  
    // This onCancel closure might execute concurrently with the operation.  
    condition.cancel()  
}
```

The operation to perform.

handler

A closure to execute on cancellation. If the task is canceled, this closure is called at most once; otherwise, it isn't called.

isolation

The actor that the operation and cancellation handler are isolated to.

Discussion

This differs from the operation cooperatively checking for cancellation and reacting to it in that the cancellation handler is *always* and *immediately* invoked when the task is canceled. For example, even if the operation is running code that never checks for cancellation, a cancellation handler still runs and provides a chance to run some cleanup code:

```
await withTaskCancellationHandler {  
    var sum = 0  
    while condition {  
        sum += 1  
    }  
    return sum  
} onCancel: {  
    // This onCancel closure might execute concurrently with the operation.  
    condition.cancel()  
}
```

Connection Negotiation

Cancellation correctness

Connection Negotiation

Cancellation correctness

Caller

The diagram consists of two horizontal bars. The top bar is red and is labeled 'Caller' to its left. The bottom bar is yellow and is labeled 'Connection' to its left. Both bars span the entire width of the slide.

Connection

Connection Negotiation

Cancellation correctness

Caller

Connection

```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller

Connection

```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller

Connection

```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller

Connection



```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller

Connection



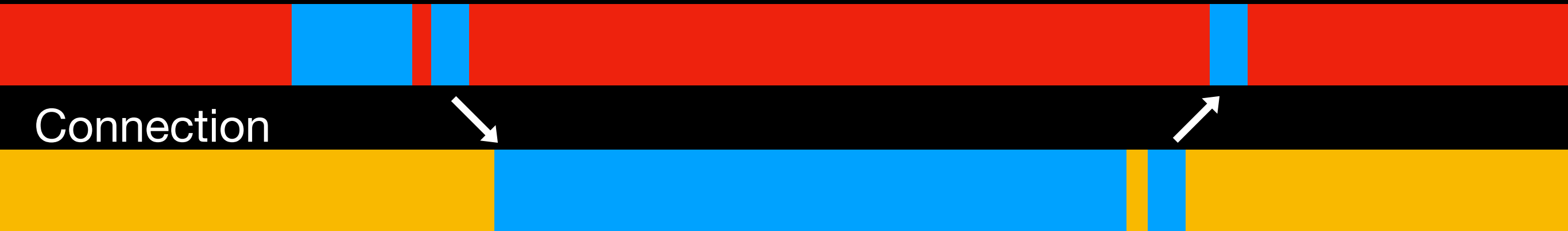
```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller

Connection



```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller

Connection

```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller

Connection

Cancellation

```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller

Connection

Cancellation



```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller

Connection

Cancellation

```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller



Connection



Cancellation



```
static func connect(remote: Remote) async throws -> Connection {  
    try await withCheckedThrowingContinuation { continuation in  
        let client = Client {  
            continuation.resume(with: $0)  
        }  
        withTaskCancellationHandler(operation: {  
            client.connect(remote: remote)  
        }, onCancel: {  
            client.cancel()  
            continuation.resume(throwing: CancellationError())  
        })  
    }  
}
```

Connection Negotiation

Cancellation correctness

Caller



Connection



Cancellation



```
static func connect(remote: Remote) async throws -> Connection {  
    try await withCheckedThrowingContinuation { continuation in  
        let client = Client {  
            continuation.resume(with: $0)  
        }  
        withTaskCancellationHandler(operation: {  
            client.connect(remote: remote)  
        }, onCancel: {  
            client.cancel()  
            continuation.resume(throwing: CancellationError())  
        })  
    }  
}
```

Connection Negotiation

Cancellation correctness

Caller

Connection

Cancellation

```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller

Connection

Cancellation

```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller



Connection



Cancellation



```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller



Connection



Cancellation



```
static func connect(remote: Remote) async throws -> Connection {  
    try await withCheckedThrowingContinuation { continuation in  
        let client = Client {  
            continuation.resume(with: $0)  
        }  
        withTaskCancellationHandler(operation: {  
            client.connect(remote: remote)  
        }, onCancel: {  
            client.cancel()  
            continuation.resume(throwing: CancellationError())  
        })  
    }  
}
```

Connection Negotiation

Cancellation correctness

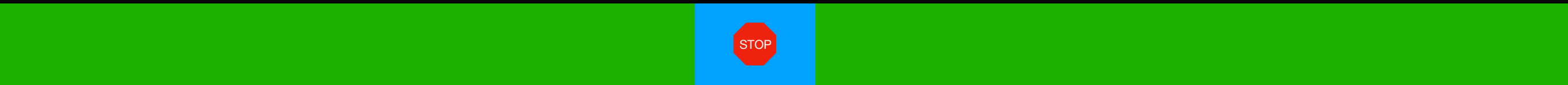
Caller



Connection



Cancellation



```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

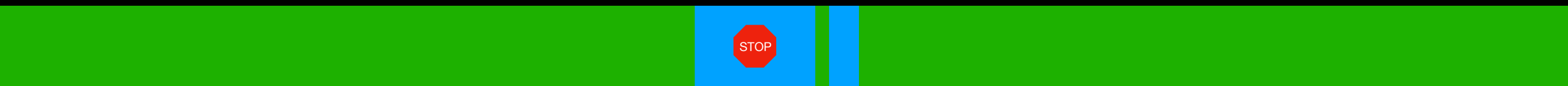
Caller



Connection



Cancellation



```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

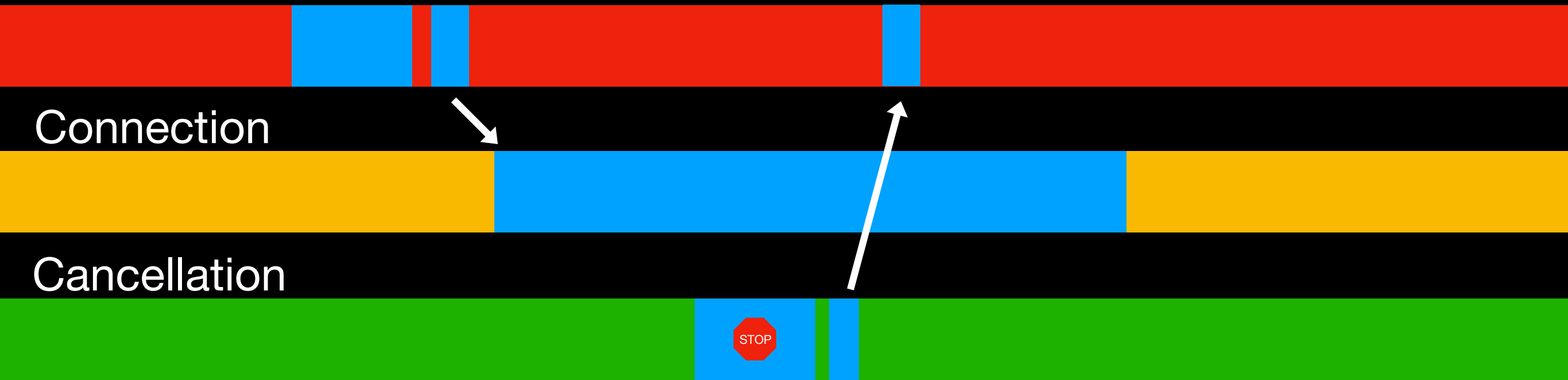
Connection Negotiation

Cancellation correctness

Caller

Connection

Cancellation



```
static func connect(remote: Remote) async throws -> Connection {  
    try await withCheckedThrowingContinuation { continuation in  
        let client = Client {  
            continuation.resume(with: $0)  
        }  
        withTaskCancellationHandler(operation: {  
            client.connect(remote: remote)  
        }, onCancel: {  
            client.cancel()  
            continuation.resume(throwing: CancellationError())  
        })  
    }  
}
```

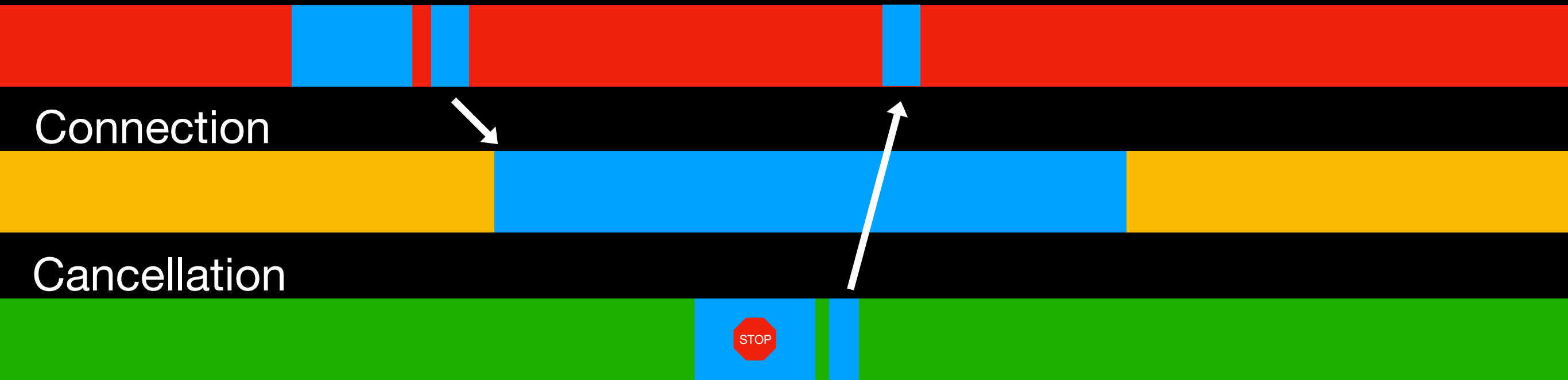
Connection Negotiation

Cancellation correctness

Caller

Connection

Cancellation



```
static func connect(remote: Remote) async throws -> Connection {  
    try await withCheckedThrowingContinuation { continuation in  
        let client = Client {  
            continuation.resume(with: $0)  
        }  
        withTaskCancellationHandler(operation: {  
            client.connect(remote: remote)  
        }, onCancel: {  
            client.cancel()  
            continuation.resume(throwing: CancellationError())  
        })  
    }  
}
```

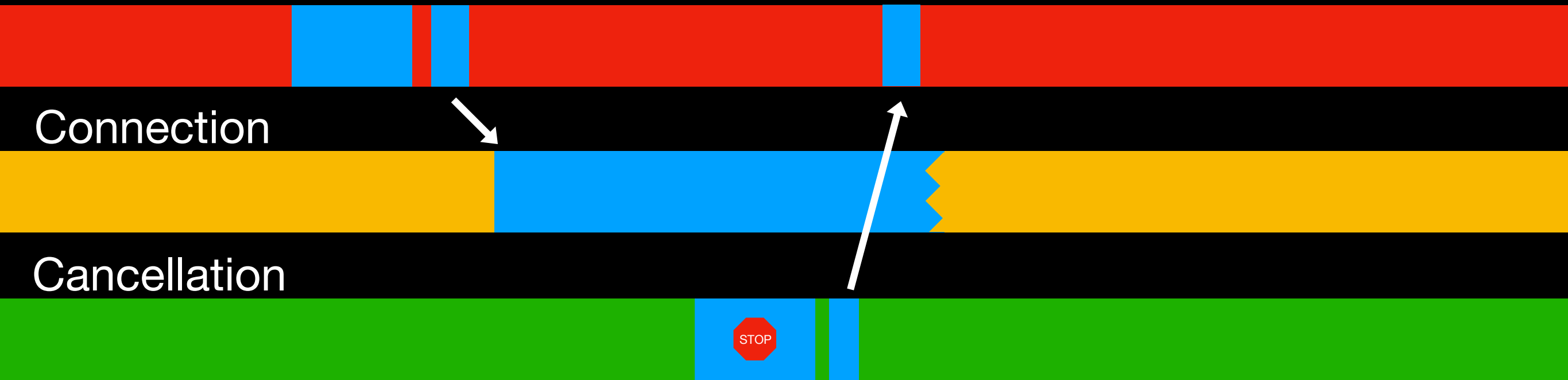
Connection Negotiation

Cancellation correctness

Caller

Connection

Cancellation



```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller

Connection

Cancellation

```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller

Connection

Cancellation

```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller



Connection



Cancellation



```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller



Connection



Cancellation



```
static func connect(remote: Remote) async throws -> Connection {  
    try await withCheckedThrowingContinuation { continuation in  
        let client = Client {  
            continuation.resume(with: $0)  
        }  
        withTaskCancellationHandler(operation: {  
            client.connect(remote: remote)  
        }, onCancel: {  
            client.cancel()  
            continuation.resume(throwing: CancellationError())  
        })  
    }  
}
```

Connection Negotiation

Cancellation correctness

Caller

Connection

Cancellation

```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller

Connection

Cancellation

```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller

Connection

Cancellation

```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```

Connection Negotiation

Cancellation correctness

Caller

Connection

Cancellation

```
static func connect(remote: Remote) async throws -> Connection {
    try await withCheckedThrowingContinuation { continuation in
        let client = Client {
            continuation.resume(with: $0)
        }
        withTaskCancellationHandler(operation: {
            client.connect(remote: remote)
        }, onCancel: {
            client.cancel()
            continuation.resume(throwing: CancellationError())
        })
    }
}
```



Connection Negotiation

Cancellation correctness

- Keep track of whether we have cancelled or not
 - Careful bookkeeping of state transitions
 - Requires synchronization between isolation domains
- What if we looked at the system API a little differently?
 - Looks almost like...something familiar
 - What is missing?

Connection Negotiation

New system API, revisited

```
protocol Client {  
    init(callback: @escaping (Result<Connection, Error>) -> Void)  
    func connect(remote: Remote)  
    func cancel()  
}
```

The screenshot shows a web browser window displaying the Apple Developer documentation for AsyncStream. The browser's address bar shows the URL `developer.apple.com/documentation/swift/asyncstream`. The page title is "Documentation" and the language is set to "Swift". The left sidebar shows a navigation menu with "All Technologies" and "Swift". Under "Swift", the "AsyncStream" section is expanded, showing a list of topics including "Creating a Continuation-Based ...", "init(Element.Type, bufferingPol...", "AsyncStream.Continuation.Buf...", "AsyncStream.Continuation", "Creating a Stream from an Asyn...", "init(unfolding: () async -> Elem...", "Finding Elements", "func contains(Self.Element) as...", "func contains(where: (Self.Ele...", and "func allSatisfy((Self.Element) a". The main content area is titled "Adapting Existing Code to Use Streams". It contains two paragraphs of text and a code block. The first paragraph explains how to adapt existing callback code to use `async-await` by using the `yield(_:)` method. The second paragraph describes a hypothetical `QuakeMonitor` type that provides callers with `Quake` instances every time it detects an earthquake. The code block shows a Swift class `QuakeMonitor` with a `quakeHandler` property and `startMonitoring()` and `stopMonitoring()` methods.

developer.apple.com/documentation/swift/asyncstream

Documentation Language: Swift

< All Technologies

Swift

▼ S AsyncStream

- Creating a Continuation-Based ...
- M init(Element.Type, bufferingPol...
- > E AsyncStream.Continuation.Buf...
- > S AsyncStream.Continuation
- Creating a Stream from an Asyn...
- M init(unfolding: () async -> Elem...
- Finding Elements
- M func contains(Self.Element) as...
- M func contains(where: (Self.Ele...
- M func allSatisfy((Self.Element) a

Adapting Existing Code to Use Streams

To adapt existing callback code to use `async-await`, use the callbacks to provide values to the stream, by using the continuation's `yield(_:)` method.

Consider a hypothetical `QuakeMonitor` type that provides callers with `Quake` instances every time it detects an earthquake. To receive callbacks, callers set a custom closure as the value of the monitor's `quakeHandler` property, which the monitor calls back as necessary.

```
class QuakeMonitor {  
    var quakeHandler: ((Quake) -> Void)?  
  
    func startMonitoring() {...}  
    func stopMonitoring() {...}  
}
```

To adapt this to use `async-await`, extend the `QuakeMonitor` to add a `quakes` property of type

Connection Negotiation

New system API, revisited

```
protocol Client {  
    init(callback: @escaping (Result<Connection, Error>) -> Void)  
    func connect(remote: Remote)  
    func cancel()  
}
```

Connection Negotiation

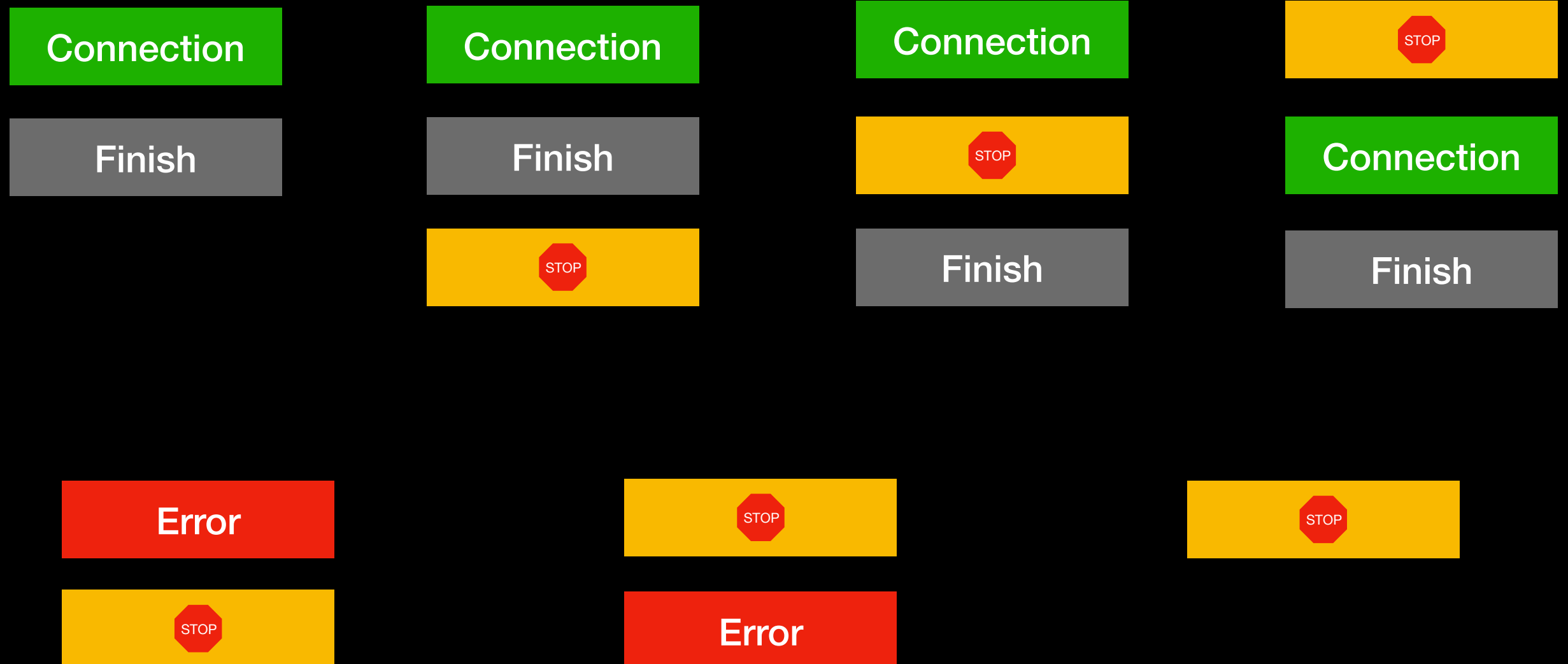
async API with cancellation, revisited

```
protocol Client {
    init(callback: @escaping (Result<Connection, Error>) -> Void)
    func connect(remote: Remote)
    func cancel()
}

extension Client {
    static func connect(remote: Remote) async throws -> Connection {
        let stream = AsyncThrowingStream { continuation in
            let client = Client {
                continuation.yield(with: $0)
                continuation.finish()
            }
            continuation.onTermination = { _ in
                continuation.finish(throwing: CancellationError())
                client.cancel()
            }
        }
        return try await stream.first(where: { _ in true })!
    }
}
```

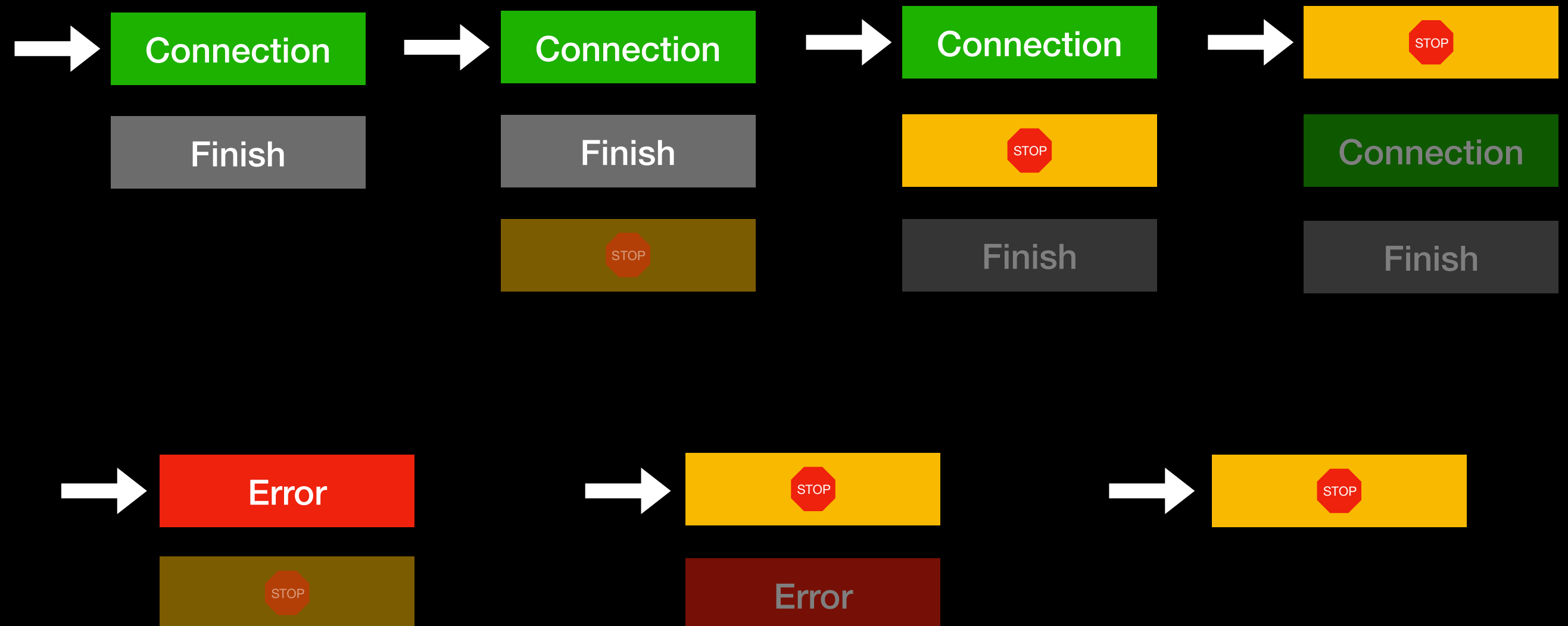
Connection Negotiation

Cancellation correctness, revisited



Connection Negotiation

Cancellation correctness, revisited



Connection Negotiation

Takeaways

- Use tools that reduce the state space you have to check
 - Serializing execution is a good way to do this
 - Sometimes more general code can be simpler
 - Consider options that perform worse if they are easier to use
- Aim for eventual correctness
 - Scheduling and execution are hard to predict
 - Most fine-grained tools will not be available

Sending Messages

Sending Messages

System API

```
protocol Connection {  
    func send(message: Message, completion: (Result<Response, Error>) -> Void)  
}
```

Sending Messages

async API

```
protocol Connection {  
    func send(message: Message, completion: (Result<Response, Error>) -> Void)  
}  
  
extension Connection {  
    func send(message: Message) async throws -> Response  
}
```

Sending Messages

async API

```
protocol Connection {  
    func send(message: Message, completion: (Result<Response, Error>) -> Void)  
}  
  
extension Connection {  
    func send(message: Message) async throws -> Response  
}
```

Sending Messages

async API

```
protocol Connection {  
    func send(message: Message, completion: (Result<Response, Error>) -> Void)  
}  
  
extension Connection {  
    func send(message: Message) async throws -> Response  
  
}
```

Sending Messages

async API

```
protocol Connection {  
    func send(message: Message, completion: (Result<Response, Error>) -> Void)  
}  
  
extension Connection {  
    func send(message: Message) async throws -> Response {  
        try await withCheckedThrowingContinuation { continuation in  
            send(message: message) {  
                continuation.resume(with: $0)  
            }  
        }  
    }  
}
```

Sending Messages

Usage

```
protocol Connection {
    func send(message: Message, completion: (Result<Response, Error>) -> Void)
}

extension Connection {
    func send(message: Message) async throws -> Response {
        try await withCheckedThrowingContinuation { continuation in
            send(message: message) {
                continuation.resume(with: $0)
            }
        }
    }
}

let connection = try await Client.connect(remote: "Saagars-MacBook-Pro")
async let responseA = connection.send(message: MessageA())
async let responseB = connection.send(message: MessageB())
print(try await (responseA, responseB))
```

Sending Messages

Correctness

- What does the previous API guarantee?
 - Messages are delivered
 - Errors are thrown
- Any other requirements?
 - Ordering

Sending Messages

Correctness

```
actor Actor {  
    func sendA() async throws -> Response {  
        print("sending A")  
        return try await connection.send(message: MessageA())  
    }  
  
    func sendB() async throws -> Response {  
        print("sending B")  
        return try await connection.send(message: MessageB())  
    }  
}  
  
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

```
actor Actor {  
    func sendA() async throws -> Response {  
        print("sending A")  
        return try await connection.send(message: MessageA())  
    }  
  
    func sendB() async throws -> Response {  
        print("sending B")  
        return try await connection.send(message: MessageB())  
    }  
}
```

```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

```
$ swift test.swift
```

```
sending B
```

```
sending A
```

```
$ ssh server "journalctl -u test"
```

```
received A
```

```
received B
```

Sending Messages

Correctness

```
actor Actor {  
    func sendA() async throws -> Response {  
        print("sending A")  
        return try await connection.send(message: MessageA())  
    }  
  
    func sendB() async throws -> Response {  
        print("sending B")  
        return try await connection.send(message: MessageB())  
    }  
}
```

```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

```
$ swift test.swift
```

```
sending B
```

```
sending A
```

```
$ ssh server "journalctl -u test"
```

```
received A
```

```
received B
```

Sending Messages

Correctness

```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

Default



Actor



```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

Default



Actor



```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

Default



Actor



```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

Default

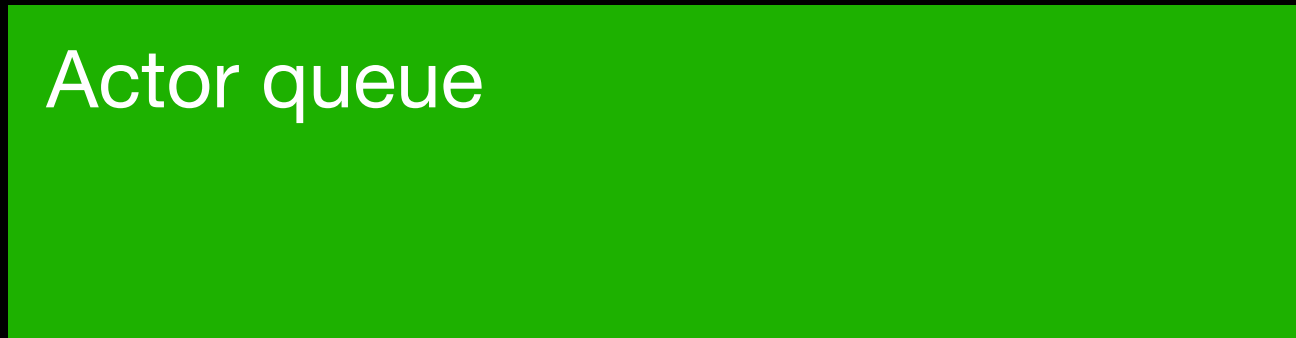


Actor



|

Actor queue



```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

Default



Actor



|

Actor queue

sendA

```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

Default



Actor



|

Actor queue

sendA

sendB

```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

Default



Actor



|

Actor queue

sendA

sendB

```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

Default



Actor



|

Actor queue

sendA

sendB

```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

Default



Actor



|

Actor

sendA

sendB

```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

Default



Actor



|

Actor “bag”

sendA

sendB

```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

Default



Actor



|

Actor "bag"



```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

Default



Actor



|

Actor “bag”



Concurrent pool



```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

Default



Actor



|

Actor “bag”

Concurrent pool “bag”

Concurrent pool

|

```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

Default



Actor

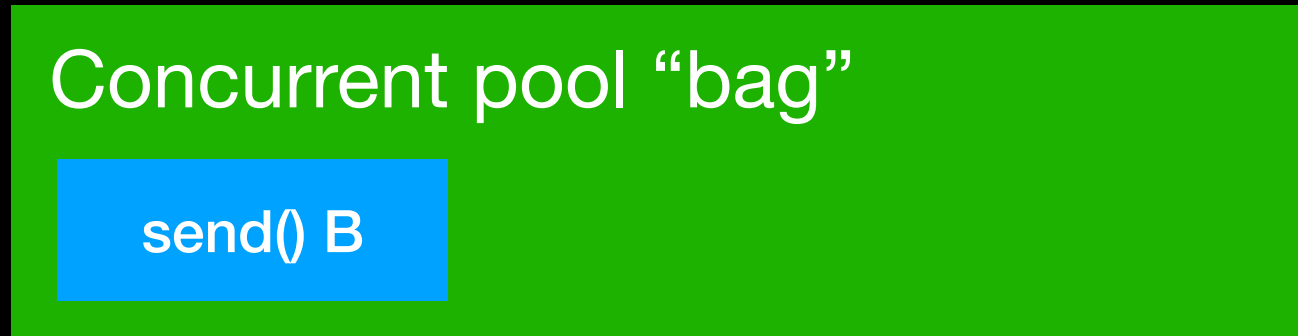


|

Actor “bag”



Concurrent pool “bag”



Concurrent pool

|



```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

Default



Actor

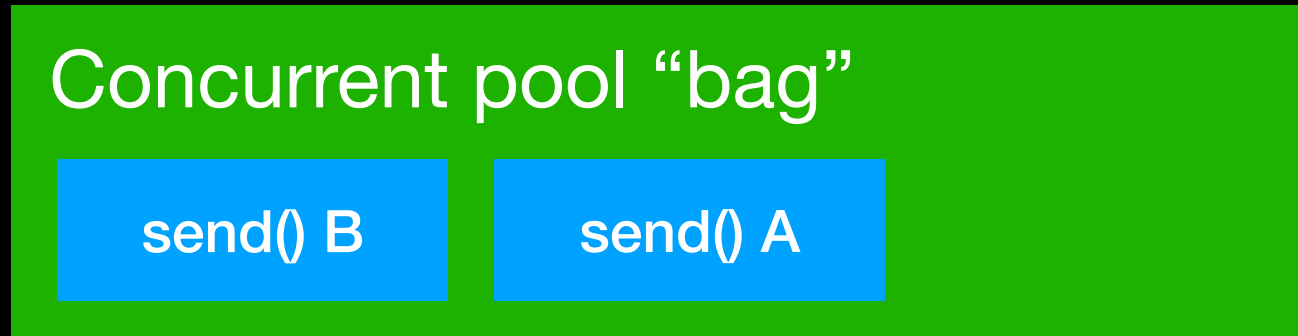


|

Actor “bag”



Concurrent pool “bag”



Concurrent pool

|



```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

Default



Actor

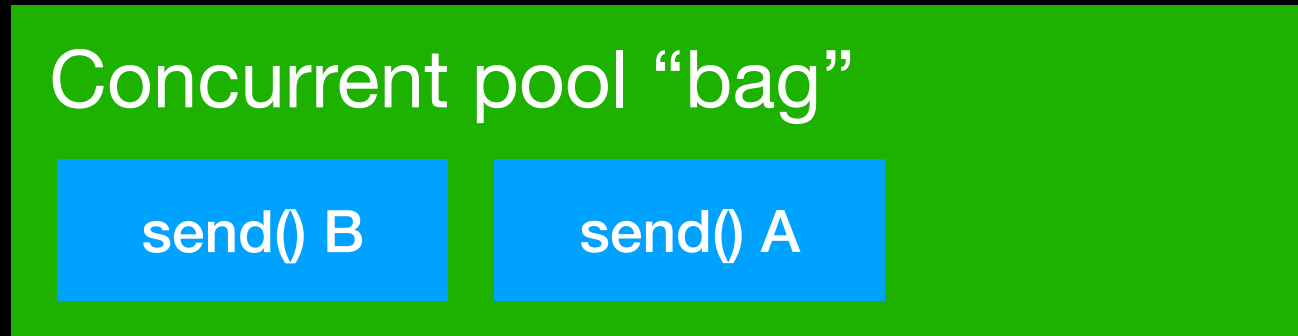


|

Actor “bag”



Concurrent pool “bag”



Concurrent pool

|



```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

Default



Actor



|

Actor “bag”

Concurrent pool “bag”

Concurrent pool



|

```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

```
protocol Connection {  
    func send(message: Message, completion: (Result<Response, Error>) -> Void)  
}  
  
extension Connection {  
    func send(message: Message) async throws -> Response {  
        try await withCheckedThrowingContinuation { continuation in  
            send(message: message) {  
                continuation.resume(with: $0)  
            }  
        }  
    }  
}
```

Sending Messages

Correctness

```
protocol Connection {  
    func send(message: Message, completion: (Result<Response, Error>) -> Void)  
}  
  
extension Connection {  
    func send(message: Message) async throws -> Response {  
        try await withCheckedThrowingContinuation { continuation in  
            send(message: message) {  
                continuation.resume(with: $0)  
            }  
        }  
    }  
}
```

Sending Messages

Correctness

```
protocol Connection {  
    func send(message: Message, completion: (Result<Response, Error>) -> Void)  
}  
  
extension Connection {  
    func send(  
        message: Message  
  
    ) async throws -> Response {  
        try await withCheckedThrowingContinuation { continuation in  
            send(message: message) {  
                continuation.resume(with: $0)  
            }  
        }  
    }  
}
```

Sending Messages

Correctness

```
protocol Connection {  
    func send(message: Message, completion: (Result<Response, Error>) -> Void)  
}  
  
extension Connection {  
    func send(  
        message: Message,  
        isolation: isolated (any Actor)? = #isolation  
    ) async throws -> Response {  
        try await withCheckedThrowingContinuation { continuation in  
            send(message: message) {  
                continuation.resume(with: $0)  
            }  
        }  
    }  
}
```

Sending Messages

Correctness

```
actor Actor {  
    func sendA() async throws -> Response {  
        defer { print("received A response") }  
        return try await connection.send(message: MessageA())  
    }  
  
    func sendB() async throws -> Response {  
        defer { print("received B response") }  
        return try await connection.send(message: MessageB())  
    }  
}  
  
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

Sending Messages

Correctness

```
actor Actor {  
    func sendA() async throws -> Response {  
        defer { print("received A response") }  
        return try await connection.send(message: MessageA())  
    }  
  
    func sendB() async throws -> Response {  
        defer { print("received B response") }  
        return try await connection.send(message: MessageB())  
    }  
}
```

```
let actor = Actor()  
Task { try await actor.sendA() }  
Task { try await actor.sendB() }
```

```
$ swift test.swift  
received B response  
received A response  
$ ssh server "journalctl -u test"  
responding for A  
responding for B
```

Sending Messages

Correctness

Sending Messages

Correctness

Connection

A solid red horizontal bar spanning the width of the slide.

Actor

A solid yellow horizontal bar spanning the width of the slide.

Sending Messages

Correctness

Connection

Actor

Connection queue

Actor “bag”

```
func send(message: Message) async throws -> Response {  
    try await withCheckedThrowingContinuation { continuation in  
        send(message: message) {  
            continuation.resume(with: $0)  
        }  
    }  
}
```

Sending Messages

Correctness

Connection

Actor

Connection queue

A response

B response

Actor “bag”

```
func send(message: Message) async throws -> Response {  
    try await withCheckedThrowingContinuation { continuation in  
        send(message: message) {  
            continuation.resume(with: $0)  
        }  
    }  
}
```

Sending Messages

Correctness

Connection

A response

Actor

Connection queue

B response

Actor “bag”

```
func send(message: Message) async throws -> Response {  
    try await withCheckedThrowingContinuation { continuation in  
        send(message: message) {  
            continuation.resume(with: $0)  
        }  
    }  
}
```

Sending Messages

Correctness

Connection

A response

Actor

Connection queue

B response

Actor “bag”

send() A

```
func send(message: Message) async throws -> Response {  
    try await withCheckedThrowingContinuation { continuation in  
        send(message: message) {  
            continuation.resume(with: $0)  
        }  
    }  
}
```

Sending Messages

Correctness

Connection

A response

B response

Actor

Connection queue

Actor “bag”

send() A

```
func send(message: Message) async throws -> Response {  
    try await withCheckedThrowingContinuation { continuation in  
        send(message: message) {  
            continuation.resume(with: $0)  
        }  
    }  
}
```

Sending Messages

Correctness

Connection

A response

B response

Actor

Connection queue

Actor “bag”

send() A

send() B

```
func send(message: Message) async throws -> Response {  
    try await withCheckedThrowingContinuation { continuation in  
        send(message: message) {  
            continuation.resume(with: $0)  
        }  
    }  
}
```

Sending Messages

Correctness

Connection

A response

B response

Actor

Connection queue

Actor “bag”

send() A

send() B

```
func send(message: Message) async throws -> Response {  
    try await withCheckedThrowingContinuation { continuation in  
        send(message: message) {  
            continuation.resume(with: $0)  
        }  
    }  
}
```

Sending Messages

Correctness

Connection

A response

B response

Actor

Connection queue

Actor “bag”

send() A

send() B

```
func send(message: Message) async throws -> Response {  
    try await withCheckedThrowingContinuation { continuation in  
        send(message: message) {  
            continuation.resume(with: $0)  
        }  
    }  
}
```

Sending Messages

Correctness

Connection

A response

B response

Actor

send() B

Connection queue

Actor “bag”

send() A

```
func send(message: Message) async throws -> Response {  
    try await withCheckedThrowingContinuation { continuation in  
        send(message: message) {  
            continuation.resume(with: $0)  
        }  
    }  
}
```

Sending Messages

Correctness

Connection

A response

B response

Actor

send() B

send() A

Connection queue

Actor “bag”

```
func send(message: Message) async throws -> Response {  
    try await withCheckedThrowingContinuation { continuation in  
        send(message: message) {  
            continuation.resume(with: $0)  
        }  
    }  
}
```

Sending Messages

Correctness

- Replies come in sequence, but we yield to the “bag”
 - Ordering can't be preserved across these calls
 - Requires serialization
- Key observation: critical region between resume and schedule
 - Can we temporarily prevent another resume?

Sending Messages

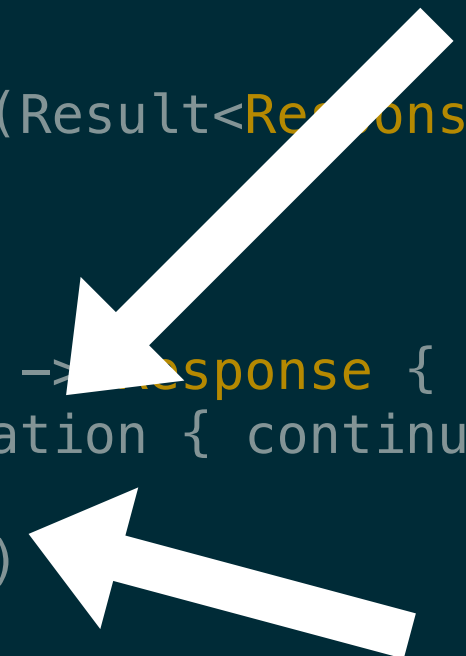
Correctness

```
                                defer { lock.unlock() }

protocol Connection {
    func send(message: Message, completion: (Result<Response, Error>) -> Void)
}

extension Connection {
    func send(message: Message) async throws -> Response {
        try await withCheckedThrowingContinuation { continuation in
            send(message: message) {
                continuation.resume(with: $0)
            }
        }
    }
}

lock.lock()
```



Sending Messages

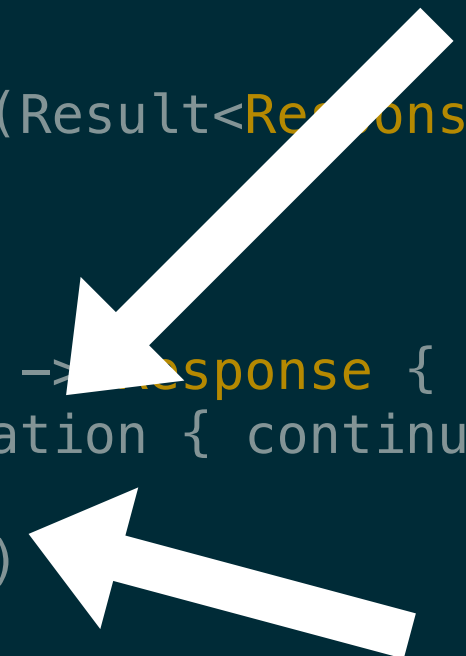
Correctness

```
                                defer { lock.unlock() }

protocol Connection {
    func send(message: Message, completion: (Result<Response, Error>) -> Void)
}

extension Connection {
    func send(message: Message) async throws -> Response {
        try await withCheckedThrowingContinuation { continuation in
            send(message: message) {
                continuation.resume(with: $0)
            }
        }
    }
}

protocol Lock {
    func lock()
    func unlock()
}
```



lock.lock()

Sending Messages

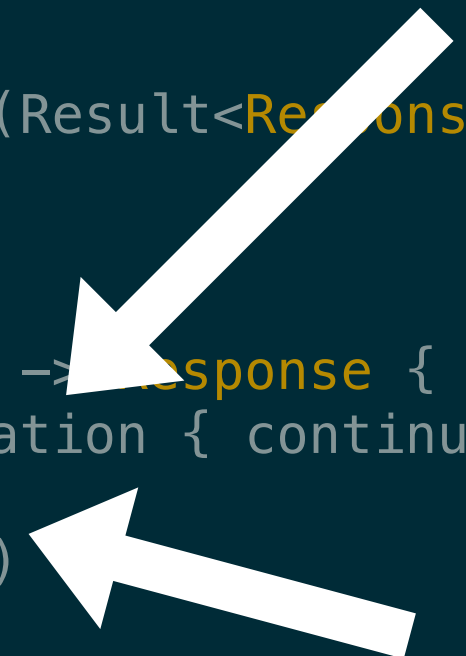
Correctness

```
                                defer { lock.unlock() }

protocol Connection {
    func send(message: Message, completion: (Result<Response, Error>) -> Void)
}

extension Connection {
    func send(message: Message) async throws -> Response {
        try await withCheckedThrowingContinuation { continuation in
            send(message: message) {
                continuation.resume(with: $0)
            }
        }
    }
}

protocol Lock {
    func lock() async
    func unlock()
}
```



lock.lock()

Sending Messages

Takeaways

- The shape of an API is not enough to define its semantics
- Swift Concurrency runtime has a very different execution model
 - Even “mechanical” migrations may break your code
- Consider serializing execution if you need ordering guarantees
- Eventual correctness can lead to very different requirements
 - Backpressure itself is a complicated topic

Questions?